14 Multi-Objective Optimization

All the optimization problems we discussed so far have only a single objective. In reality, we often have to optimize multiple objectives simultaneously. For example, we may want to improve the performance of a product while trying to minimize the cost at the same time. In this case, we are dealing with a multi-objective optimization problem. Many new concepts are required for solving multi-objective optimization. Furthermore, these multi-objectives can be conflicting, and thus some trade-offs are needed. As a result, a set of Pareto-optimal solutions, rather than a single solution, must be found. This often requires multiple runs of solution algorithms.

14.1 Multi-Objective Optimization

The optimization problem with a single objective that we have discussed so far can be considered a scalar optimization problem because the objective function always reaches a single global optimal value or a scalar. For multi-objective optimization, the multiple-objective functions form a vector, and thus it is also called *vector optimization* [3,6,9,10].

Any multi-objective optimization problem can generally be written as

$$\underset{\boldsymbol{x}\in\Re^d}{\text{Minimize}}\ \boldsymbol{f}(\boldsymbol{x}) = [f_1(\boldsymbol{x}),\ f_2(\boldsymbol{x}),\ \ldots,\ f_M\ (\boldsymbol{x})],$$

$$\text{subject to} \quad g_j(\boldsymbol{x}) \le 0, \quad j = 1, 2, \ldots, J, \tag{14.1}$$

$$h_k(\boldsymbol{x}) = 0, \quad k = 1, 2, \ldots, K, \tag{14.2}$$

where $\boldsymbol{x} = (x_1, x_2, \ldots, x_d)^T$ is the vector of decision variables. In some formulations used in the optimization literature, inequalities $g_j (j = 1, \ldots, J)$ can also include any equalities, because an equality $\phi(\boldsymbol{x}) = 0$ can be converted into two inequalities $\phi(\boldsymbol{x}) \le 0$ and $\phi(\boldsymbol{x}) \ge 0$. However, for clarity, here we list the equalities and inequalities separately.

The space $\mathcal{F} = \Re^d$ spanned by the vectors of decision variables $\boldsymbol{x}$ is called the *search space*. The space $\mathcal{S} = \Re^M$ formed by all the possible values of objective functions is called the *solution space* or *objective space*. Comparing with the single-objective function whose solution space is (at most) $\Re$, the solution space for multi-objective optimization is considerably larger. In addition, because we know that we are dealing with multi-objectives $\boldsymbol{f}(\boldsymbol{x}) = [f_i]$, for simplicity we can write f_i as $f(\boldsymbol{x})$ without causing any confusion.

Nature-Inspired Optimization Algorithms. http://dx.doi.org/10.1016/B978-0-12-416743-8.00014-2
© 2014 Elsevier Inc. All rights reserved.

A multi-objective optimization problem, unlike a single-objective optimization problem, does not necessarily have an optimal solution that minimizes all the multi-objective functions simultaneously. Often, objectives may conflict with each other, and the optimal parameters of some objectives usually do not lead to the optimality of other objectives (sometimes they even make them worse). For example, we want first-class-quality service on our holidays, but at the same time we want to pay as little as possible. The high-quality service (one objective) will inevitably cost much more, but this is in conflict with the other objective (to minimize cost).

Therefore, among these often conflicting objectives, we have to choose some trade-off or achieve a certain balance of objectives. If none of these is possible, we must choose a list of preferences as to which objectives should be achieved first. More important, we have to compare different objectives and make a compromise. This usually requires a reformulation, and one of the most popular approaches to reformulation is to find a scalar-valued function that represents a weighted combination or preference order of all objectives. Such a scalar function is often referred to as the *preference function* or *utility function*. A simple way to construct this scalar function is to use the weighted sum

$$\Phi(f_1(\boldsymbol{x}), \ldots, f_M(\boldsymbol{x})) = \sum_{i=1}^{M} w_i f_i(\boldsymbol{x}), \tag{14.3}$$

where w_i are the weighting coefficients.

Naïvely, some may wonder what happens if we try to optimize each objective individually so that each will achieve the best (the minimum for a minimization problem). In this case, we have

$$F^* = (f_1^*, f_2^*, \ldots, f_M^*), \tag{14.4}$$

which is called the *ideal objective* vector. However, there is no solution that corresponds to this ideal vector. That is to say, it is a nonexistent solution. The only exception is when all the objectives correspond to the same solution, and in this case, these multi-objectives are not conflicting, leading to the case when the Pareto front typically collapses into a single point [4].

For multi-objective optimization, we have to introduce some new concepts related to Pareto optimality.

14.2 Pareto Optimality

A vector $\boldsymbol{u} = (u_1, .., u_d)^T \in \mathcal{F}$, is said to dominate another vector $\boldsymbol{v} = (v_1, \ldots, v_d)^T$ if and only if $u_i \leq v_i$ for $\forall i \in \{1, \ldots, d\}$ and $\exists i \in \{1, \ldots, d\} : u_i < v_i$. This "partial-less" or component-wise relationship is denoted by

$$\boldsymbol{u} \prec \boldsymbol{v}, \tag{14.5}$$

which is equivalent to

$$\forall i \in \{1, \ldots, d\} : u_i \leq v_i \wedge \exists i \in \{1, \ldots, d\} : u_i < v_i. \tag{14.6}$$

Here $\wedge$ means the logical *and*. In other words, no component of $\boldsymbol{u}$ is larger than the corresponding component of $\boldsymbol{v}$, and at least one component is smaller. Similarly, we can define another dominance relationship $\preceq$ by

$$\boldsymbol{u} \preceq \boldsymbol{v} \Longleftrightarrow \boldsymbol{u} \prec \boldsymbol{v} \vee \boldsymbol{u} = \boldsymbol{v}. \tag{14.7}$$

Here $\vee$ means *or*. It is worth pointing out that for maximization problems, the dominance can be defined by replacing $\prec$ with $\succ$.

A point or a solution $\boldsymbol{x}_* \in \Re^d$ is called a *Pareto-optimal solution* or non-inferior solution to the optimization problem if there is no $\boldsymbol{x} \in \Re^d$ satisfying $f_i(\boldsymbol{x}) \leq f_i(\boldsymbol{x}_*)$, $(i = 1, 2, \ldots, M)$. In other words, $\boldsymbol{x}_*$ is Pareto-optimal if there exists no feasible vector (of decision variables in the search space) that would decrease some objectives without simultaneously causing an increase in at least one other objective. That is to say, optimal solutions are solutions that are not dominated by any other solutions. When mapping to objective vectors, they represent different trade-offs between multiple objectives.

Furthermore, a point $\boldsymbol{x}_* \in \mathcal{F}$ is called a *nondominated* solution if no solution can be found that dominates it. A vector is called *ideal* if it contains the decision variables that correspond to the optima of objectives when each objective is considered separately.

Unlike the single-objective optimization with often a single optimal solution, multi-objective optimization will lead to a set of solutions, called the Pareto-optimal set $\mathcal{P}^*$, and the decision vectors $\boldsymbol{x}_*$ for this solution set are thus called nondominated. That is to say, the set of optimal solutions in the decision space forms the Pareto (optimal) set. The image of this Pareto set in the objective or response space is called the *Pareto front*. In the literature, the set $\boldsymbol{x}_*$ in the decision space that corresponds to the Pareto-optimal solutions is also called an *efficient set*. The set (or plot) of the objective functions of these nondominated decision vectors in the Pareto-optimal set forms the so-called Pareto front $\mathcal{P}$ or Pareto frontier.

In short, $\boldsymbol{u} \preceq \boldsymbol{v}$ means that $\boldsymbol{u}$ dominates $\boldsymbol{v}$; that is, $\boldsymbol{u}$ is nondominated by $\boldsymbol{v}$, or $\boldsymbol{v}$ is dominated by $\boldsymbol{u}$. This definition may be too theoretical. To put it in practical terms, $\boldsymbol{u}$ is noninferior to $\boldsymbol{v}$ (i.e., $\boldsymbol{u}$ is better or no worse than $\boldsymbol{v}$). Intuitively, when $\boldsymbol{u}$ dominates $\boldsymbol{v}$, we can loosely say that $\boldsymbol{u}$ is better than $\boldsymbol{v}$. The domination concept provides a good way to compare solutions for multi-objective optimization, and the aim of multi-objective optimization is to find such nondominated solutions. For any two solution vectors $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, there are only three possibilities: $\boldsymbol{x}_1$ dominates $\boldsymbol{x}_2$, or $\boldsymbol{x}_2$ dominates $\boldsymbol{x}_1$, or $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ do not dominate each other. Among many interesting properties of domination, transitivity still holds. That is, if $\boldsymbol{x}_1$ dominates $\boldsymbol{x}_2$, and $\boldsymbol{x}_2$ dominates $\boldsymbol{x}_3$, then $\boldsymbol{x}_1$ dominates $\boldsymbol{x}_3$.

Using this notation, the Pareto front $\mathcal{P}$ can be defined as the set of nondominated solutions so that

$$\mathcal{P} = \{\boldsymbol{s} \in \mathcal{S} \,\Big|\, \nexists \boldsymbol{s}' \in \mathcal{S} : \boldsymbol{s}' \prec \boldsymbol{s}\}, \tag{14.8}$$

or, in terms of the Pareto-optimal set in the search space,

$$\mathcal{P}^* = \{\boldsymbol{x} \in \mathcal{F} \,\Big|\, \nexists \boldsymbol{x}' \in \mathcal{F} : \boldsymbol{f}(\boldsymbol{x}') \prec \boldsymbol{f}(x)\}. \tag{14.9}$$

All the nondominated solutions in the whole feasible search space form the so-called globally Pareto-optimal set, which is simply referred to as the Pareto front.

The identification of the Pareto front is not an easy task; it often requires a parametric analysis, say, by treating all but one objective, say, f_i in an M-objective optimization problem, so that f_i is a function of $f_1, \ldots, f_{i-1}, f_{i+1}, \ldots$, and f_M. By maximizing the f_i when varying the values of the other $M-1$ objectives, there can be enough solution points that will trace out the Pareto front.

Example 14.1. For example, we have four Internet service providers, A, B, C, and D. We have two objectives by which customers choose their service: (1) as cheap as possible, and (2) higher bandwidth. The providers' details are listed here:

IP provider	Cost (£/month)	Bandwidth (Mb)
A	20	80
B	25	112
C	30	56
D	40	112

From the table, we know that option C is dominated by A and B because both objectives are improved (low cost and faster speed). Option D is dominated by B. Thus, solution C is an inferior solution, and so is D. Both solutions A and B are noninferior or nondominated solutions. However, deciding which solution (A or B) to choose is not easy, since provider A outperforms B on the first objective (cheaper) whereas B outperforms A on another objective (faster). In this case, we say these two solutions are *incomparable*. The set of nondominated solutions A and B forms the Pareto front, which is a mutually incomparable set.

For a minimization problem with two objectives, the basic concepts of the nondominated set, Pareto front, and ideal vectors are shown in Figure 14.1. Obviously, if we combine these two into a single composite objective, we can compare, for example, the cost per unit Mb. In this case, we essentially reformulate the problem as a scalar optimization problem. For choice A, each Mb costs £0.25, whereas it costs about £0.22 for choice B. So, we should choose B. However, in reality, we usually have many incomparable solutions, and it is often impossible to compromise in some way. In addition, the real choice depends on our preference and our emphasis on various objectives.

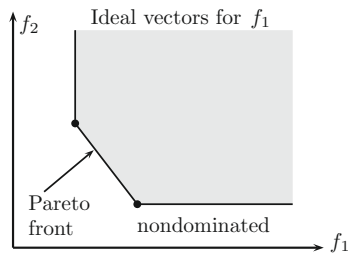


Figure 14.1 The nondominated set, Pareto front, and ideal vectors in a minimization problem with two objectives, f_1 and f_2.

It is worth pointing out that even though we have produced high-quality multiple Pareto-optimal solutions, the final choice of a point on the Pareto front is usually up to the decision makers, who have higher-level information or rules to make the decision. Such higher-level information is typically nontechnical, inaccurate, highly subjective, and often not part of the optimization problem.

Multi-objective optimization is usually difficult to solve, partly due to the lack of efficient tools and partly due to the complexity of this type of problem. Loosely speaking, there are three ways to deal with multi-objective problems: a direct approach, aggregation or transformation, and Pareto-set approximation. However, the current trends tend to be evolutionary approaches to approximating Pareto fronts [4,8].

The direct approach is difficult, especially in the case when multiple objectives seem conflicting. Therefore, we often use aggregation or transformation by combining multiple objectives into a single composite objective so that the standard methods for optimization discussed in this book can be used. We focus on this approach in the rest of this chapter. However, with this approach, the solutions typically depend on the way we combine the objectives. A third way is to try to approximate the Pareto set so as to obtain a set of mutually nondominated solutions.

To transform a multi-objective optimization problem into a single objective, we can often use the method of weighted sum and utility methods. We can also choose the most important objective of our interest as the only objective while rewriting other objectives as constraints with imposed limits.

14.3 Weighted Sum Method

The weighted sum method combines all the multi-objective functions into one scalar, composite objective function using the weighted sum

$$F(\boldsymbol{x}) = w_1 f_1(\boldsymbol{x}) + w_2 f_2(\boldsymbol{x}) + \cdots + w_M \ f_M(\boldsymbol{x}). \tag{14.10}$$

An issue arises in assigning the weighting coefficients $\boldsymbol{w} = (w_1, w_2, \ldots, w_M)$, because the solution strongly depends on the chosen weighting coefficients. Obviously, these weights have be positive, satisfying

$$\sum_{i=1}^{M} w_i = 1, \quad w_i \in (0, 1). \tag{14.11}$$

Let us first look at an example.

Example 14.2. The classical three-objective functions are commonly used for testing multi-objective optimization algorithms. These functions are

$$f_1(x, y) = x^2 + (y-1)^2, \tag{14.12}$$

$$f_2(x, y) = (x-1)^2 + y^2 + 2, \tag{14.13}$$

$$f_3(x, y) = x^2 + (y+1)^2 + 1, \tag{14.14}$$

where $(x, y) \in [-2, 2] \times [-2, 2]$.

If we combine all three functions into a single function $f(x, y)$ using the weighted sum, we have

$$f(x, y) = \alpha f_1 + \beta f_2 + \gamma f_3, \qquad \alpha + \beta + \gamma = 1. \tag{14.15}$$

The stationary point is determined by

$$\frac{\partial f}{\partial x} = 0, \qquad \frac{\partial f}{\partial y} = 0, \tag{14.16}$$

which lead to

$$2\alpha + 2\beta(x - 1) + 2\gamma = 0 \tag{14.17}$$

and

$$2\alpha(y - 1) + 2\beta y + 2\gamma(y + 1) = 0. \tag{14.18}$$

The solutions are

$$x_* = \beta, \qquad y_* = \alpha - \gamma. \tag{14.19}$$

This implies that $x_* \in [0, 1]$ and $y_* \in [-1, 1]$. Consequently, $f_1 \in [0, 5]$, $f_2 \in [2, 4]$ and $f_3 \in [1, 6]$. In addition, the solution or the optimal location varies with the weighting coefficients α, β, and γ. In the simplest case, $\alpha = \beta = \gamma = 1/3$, we have

$$x_* = \frac{1}{3}, \qquad y_* = 0. \tag{14.20}$$

This location is marked with a short, thick line in Figure 14.2.

Now the original multi-objective optimization problem has been transformed into a single-objective optimization problem. Thus, the solution methods for solving single-objective problems are all valid. For example, we can use the particle swarm

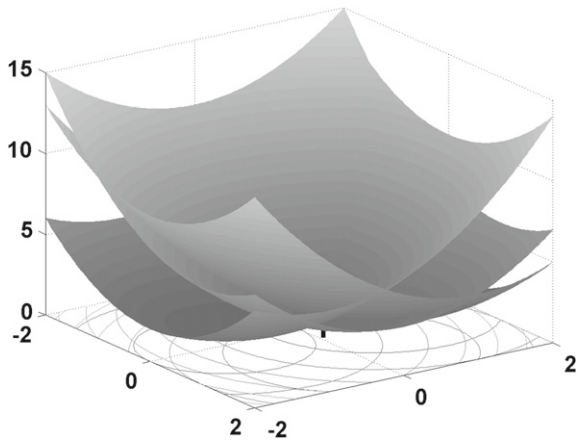


Figure 14.2 Three functions reach the global minimum at $x_* = \beta$, $y_* = \alpha - \gamma$.

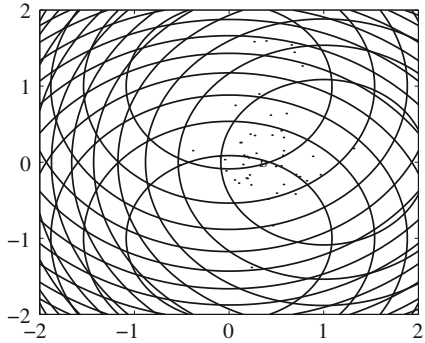


Figure 14.3 Final locations of 40 particles after five iterations. The optimal point is at $(1/3, 0)$, marked with ø.

optimization to find the optimal solution for given parameters α, β, and γ. Figure 14.3 shows the locations of 40 particles at $t = 5$ iterations. We can see that the particles converge toward the true optimal location marked with ø. Obviously, the accuracy will improve if we continue the iterations.

However, there is an important issue here. The combined weighted sum transforms the optimization problem into a single objective, which is not necessarily equivalent to the original multi-objective problem because the extra weighting coefficients could be arbitrary, whereas the final solutions still depend on these coefficients. Furthermore, there are so many ways to construct the weighted sum function, and there is no easy guideline to choose which form is the best for a given problem. When there is no rule to follow, the simplest choice obviously is to use the linear form. But there is no reason that the weighted sum should be linear. In fact, we can use other combinations such as the following quadratic weighted sum:

$$\Pi(\boldsymbol{x}) = \sum_{i=1}^{M} w_i f_i^2(\boldsymbol{x}) = w_1 f_1^2(\boldsymbol{x}) + \cdots + w_M \ f_M^2(\boldsymbol{x}) \tag{14.21}$$

and the others.

Another important issue is that of how to choose the weighting coefficients, since the solutions depend on these coefficients. Choosing weighting coefficients is essentially for decision maker(s) to assign a preference order to the multi-objectives. This leads to a more general concept of the utility function (or preference function) that reflects the preference of the decision maker(s).

Ideally, a different weight vector should result in a different trade-off point on the Pareto front; however, in reality, this is usually not the case. Different combinations of weight coefficients can lead to the same point or points very close to each other, and consequently the points are not uniformly distributed on the Pareto front. In fact, a single trade-off solution on the Pareto front just represents one sampling point, and there is no technique to ensure uniform sampling on the front. All the issues still form an active area of research.

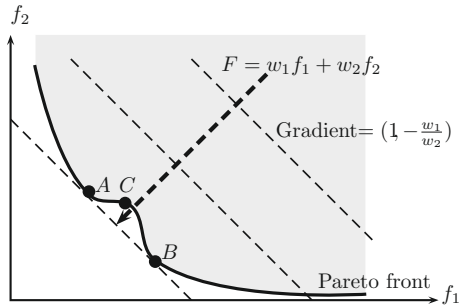


Figure 14.4 Weighted sum method for two objectives f_1 and f_2 and $w_1 + w_2 = 1$.

It is worth pointing out that the linear weighted sum method

$$\Pi(\boldsymbol{x}) = \sum_{i=1}^{M} w_i f_i(\boldsymbol{x}), \quad \sum_{i=1}^{M} w_i = 1, \quad w_i > 0, \tag{14.22}$$

only works for problems with convex Pareto fronts. As we can see from Figure 14.4, where two objectives are combined into one for a given set of $w_1 + w_2 = 1$, the composite function F is minimized. For any given set (w_1, w_2), a (dashed) line has a gradient $(1, -w_1/w_2)$ that will become tangent to the Pareto front when moving downward to the left, and that touching point is the minimum of F. However, at the nonconvex segment, if the aim is point C, the weighted sum method will usually lead to either point A or point B, depending on the values of w_1 (since $w_2 = 1 - w_1$).

The weighted sum method is one of the most widely used due to its simplicity. However, it is usually difficult to generate a good set of points that are uniformly distributed on the Pareto front. In addition, this method only works for convex Pareto fronts. Furthermore, proper scalings or normalization of the objectives are often needed so that the ranges/values of each objective should be comparable; otherwise, the weight coefficients are not well distributed and thus lead to biased sampling on the Pareto front.

For more complex multi-objective optimization problems, another widely used and yet more robust method is the ϵ-constraint method. Before we proceed, let us discuss the utility methods, which can be considered the different ways of forming composite objectives.

14.4 Utility Method

The weighted sum method is essentially a deterministic value method if we consider the weighting coefficients as the ranking coefficients. This implicitly assumes that the consequence of each ranking alternative can be characterized with certainty. This method can be used to explore the implications of alternative value judgment. The utility method, on the other hand, considers uncertainty in the criteria values for each alternative, which is a more realistic method because there is always some degree of uncertainty about the outcome of a particular alternative.

Utility (or preference) functions can be associated with risk attitude or preferences. For example, say that you are offered a choice between a guaranteed £500 and a 50/50 chance of zero or £1000. How much are you willing to pay to take the gamble? The expected payoff of each choice is £500, and thus it is fair to pay $0.5 \times 1000 + (1 - 0.5) \times 0 =$ £500 for such a gamble. A risk-seeking decision maker would risk a lower payoff in order to have a chance to win a higher prize, whereas a risk-averse decision maker would be happy with the safe choice of £500.

For a risk-neutral decision maker, the choice between a guaranteed £500 and the 50/50 gamble is indifferent, since both choices have the same expected value of £500. In reality, the risk preference can vary from person to person and may depend on the type of problem. The utility function can have many forms, and one of the simplest is the exponential utility (of representing preference)

$$u(x) = \frac{1 - e^{-(x-x_a)/\rho}}{1 - e^{-(x_b-x_a)/\rho}}, \tag{14.23}$$

where x_a and x_b are the lowest and highest levels of x, and ρ is called the *risk tolerance* of the decision maker.

The utility function defines combinations of objective values $f_1, \ldots, f_M$ that a decision maker finds equally acceptable or indifferent. So, the contours of the constant utility are referred to as the *indifference curves*. The optimization now becomes the maximization of the utility. For a maximization problem with two objectives f_1 and f_2, the idea of the utility contours (indifference curves), the Pareto front, and the Pareto solution with maximum utility (point A) are shown in Figure 14.5. When the utility function touches the Pareto front in the feasible region, it then provides a maximum utility Pareto solution (marked with A).

For two objectives f_1 and f_2, the utility function can be constructed in different ways. For example, the combined product takes the following form:

$$U(f_1, f_2) = k f_1^{\alpha} f_2^{\beta}, \tag{14.24}$$

where α and β are nonnegative exponents and k a scaling factor. On the other hand, the aggregated utility function for the same two objectives can be defined as

$$U(f_1, f_2) = \alpha f_1 + \beta f_2 + [1 - (\alpha + \beta)] f_1 \ f_2. \tag{14.25}$$

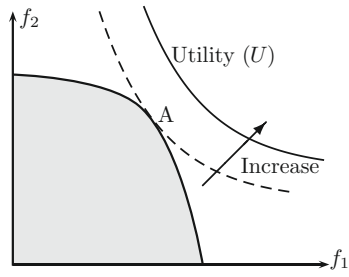


Figure 14.5 Finding the Pareto solution with maximum utility in a maximization problem with two objectives.

There are many other forms. The aim of a decision maker constructing utility functions is to form a mapping $U : \Re^p \mapsto \Re$ so that the total utility function has monotonic and/or convexity properties for easy analysis. It will also improve the quality of the Pareto solution(s) with maximum utility. Let us look at a simple example.

Example 14.3. We now try to solve the simple two-objective optimization problem:

$$\underset{(x,y)\in\Re^2}{\text{Maximize}} \; f_1(x, y) = x + y, \qquad f_2(x, y) = x,$$

subject to

$$x + \alpha y \le 5, \qquad x \ge 0, \quad y \ge 0,$$

where $0 < \alpha < 1$. Let us use the simple utility function

$$U = f_1 \, f_2,$$

which combines the two objectives. The line connecting the two corner points $(5, 0)$ and $(0, 5/\alpha)$ forms the Pareto front (see Figure 14.6). It is easy to check that the Pareto solution with the maximum utility is $U = 25$ at $A(5, 0)$ when the utility contours touch the Pareto front with the maximum possible utility.

The complexity of multi-objective optimization makes the construction of utility functions a difficult task because there are many ways to construct such functions.

14.5 The ϵ-Constraint Method

An interesting way of dealing with multi-objective optimization is to write objectives except one as constraints. Let us try to rewrite the following unconstrained optimization as a single-objective constrained optimization problem:

$$\text{Minimize } f_1(\boldsymbol{x}), \, f_2(\boldsymbol{x}), \ldots, \, f_M(\boldsymbol{x}).$$

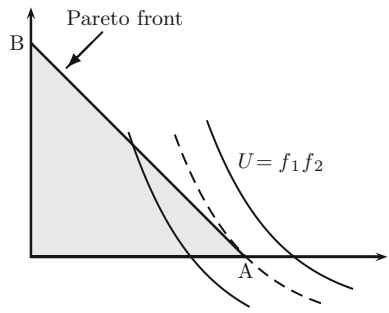


Figure 14.6 The Pareto front is the line connecting A$(5, 0)$ and B$(0, 5/\alpha)$. The Pareto solution with maximum utility is $U_* = 25$ at point A.

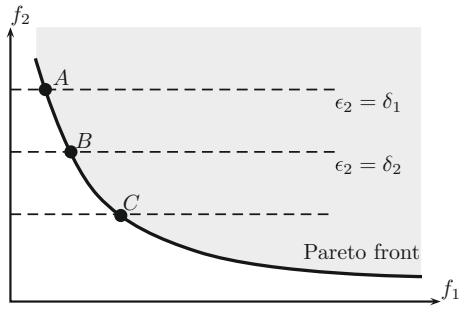


Figure 14.7 Slicing the objective domain in the ϵ-constraint method.

To achieve this goal, we often choose the most important objective of our preference, say, $f_q(\boldsymbol{x})$, as the main objective, while imposing limits on the other objectives. That is,

$$\text{Minimize} \quad f_q(\boldsymbol{x}),$$

$$\text{subject to} \quad f_i \le \epsilon_i, \quad (i = 1, 2, q-1, q+1, \ldots, M),$$

where the limits ϵ_i are given. In the simplest case, we can choose $q = 1$. Haimes et al. were probably the first to suggest this reformation method [7].

In principle, the problem can be solved using the standard optimization algorithms for single-objective optimization. In essence, this is a slicing method that splits the objective domain into different subdomains. For example, in the case of a bi-objective problem, as shown in Figure 14.7, we take f_2 as the constraint. This problem becomes

$$\text{Minimize } f_1(\boldsymbol{x}), \tag{14.26}$$

subject to

$$f_2(\boldsymbol{x}) \le \epsilon_2, \tag{14.27}$$

where ϵ_2 is a number, not necessarily small. For any given value of ϵ_2, the objective domain is split into two subdomains: $f_2 \le \epsilon_2 = \delta_1$ (feasible) and $f_2 > \epsilon_2 = \delta_1$ (infeasible). The minimization of f_1 in the feasible domain leads to the globally optimal point A. Similarly, for a different value of $\epsilon_2 = \delta_2$, the minimum of f_1 gives point B.

Let us look at a bi-objective optimization example, called *Schaffer's min-min function* [11]:

$$\text{Minimize } f_1(x) = x^2, \quad f_2(x) = (x-2)^2, \quad x \in [-10^3, 10^3]. \tag{14.28}$$

If we use f_1 as the objective and $f_2 \le \epsilon_2$ as the constraint, we can set $\epsilon_2 \in [0, 4]$ with 20 different values. Then we can solve it using a single-objective optimization technique. The 20 points of approximated Pareto-optimal solutions and the true Pareto front are shown in Figure 14.8. However, if we use f_2 as the objective and f_1 as the constraint, we follow exactly the same procedure, with the results shown in Figure 14.9.

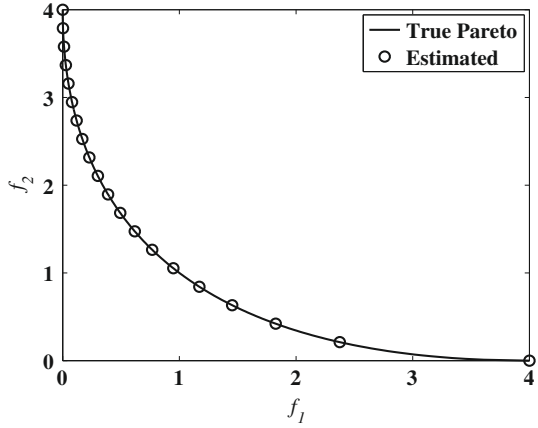


Figure 14.8 A true Pareto front and the estimated front when setting f_1 as the objective and f_2 as the constraint.

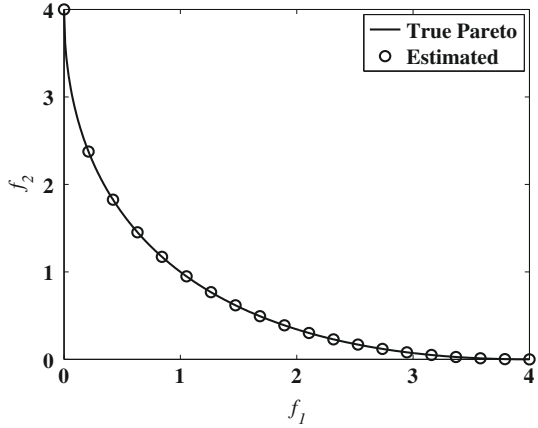


Figure 14.9 The true Pareto front and the estimated front when setting f_2 as the objective and f_1 as the constraint.

As we can see from both figures, the distributions of the approximate Pareto points are different, though they look similar.

As this example has demonstrated, the distributions of the sampling points on the Pareto front may depend on the actual formulation and the order of choosing the main objective.

The advantage of this method is that it works well for complex problems with nonconvex Pareto fronts. However, it does have some disadvantages. There could be many different formulations for choosing the main objectives and the rest of objectives as constraints. Different formulations may lead to different computational efforts. In addition, there is no guarantee that the points generated on the Pareto front are uniformly distributed, as we saw in the previous example.

Furthermore, it is difficult to impose the right range of ϵ_i. In the previous example, if we set ϵ_2 too small, say, $\epsilon_2 \to 0$, there may not be a feasible solution. On the other hand, if we set ϵ too high, it will be difficult to find the minimum of f_1 even if it exists, because the number of evaluations for this single-objective optimization problem may increase. In practice, some prior knowledge is required to impose the correct limits. Otherwise, the solutions obtained may not be the solution to the original problem.

The good news is that recent trends tend to use evolutionary approaches such as genetic algorithms. We briefly introduce some of these metaheuristic methods in the rest of this chapter.

14.6 Metaheuristic Approaches

So far, we have seen that finding solutions to a multi-objective optimization problem is usually difficult, even by the simple weighted sum method and utility function. However, there are other promising methods that work well for multi-objective optimization problems, especially metaheuristic methods such as simulated annealing, particle swarm optimization, cuckoo search, and firefly algorithms [4,14–16,18,19].

There are many potential ways to extend the standard simulated annealing to solve multi-objective optimization problems. A direct and yet simple extension is to modify the acceptance probability as the joint probability

$$p_a = \prod_{j=1}^{m} p_j = \prod_{j=1}^{m} e^{-\Delta f_j / kT_j},$$

where k is Boltzmann constant, which can be taken as $k = 1$, and $\Delta f_j = f_j(\boldsymbol{x}_n) - f_j(\boldsymbol{x}_{n-1})$ is the change of each individual objective. For details, refer to the article by Suppapitnarm et al. [13].

In 1985, Schaffer was probably the first to use vector evaluated genetic algorithms (VEGA) to solve multi-objective optimization, without using any composite aggregation, by combining all objectives into a single objective [11]. Since then, many metaheuristic algorithms such as PSO, SA, and F have been extended to solve multi-objective optimization problems successfully. Interested readers can refer to the list of more advanced literature at the end of this chapter.

The main difficulty in extending any single-objective optimization algorithm to accommodate multiple objectives is that we have to consider the dominance of each candidate solution set. In the standard PSO, we update the velocity

$$\boldsymbol{v}_i^{t+1} = \theta \boldsymbol{v}_i^t + \alpha \boldsymbol{\epsilon}_1 \cdot (\boldsymbol{g}^* - \boldsymbol{x}_i^t) + \beta \boldsymbol{\epsilon}_2 \cdot (\boldsymbol{x}_i^* - \boldsymbol{x}_i^t),$$

where $\boldsymbol{g}^*$ is the current global optimum found so far. A possible extension is to use an external repository or archive to store nondominated solutions at each iteration. Then we select such a solution $\boldsymbol{h}^*$ from this repository to replace $\boldsymbol{g}^*$. This idea was first proposed by Coello et al. [1].

There are many evolutionary approaches to multi-objective optimization, and many variants and new approaches are under active research [12]. For a relative comprehensive review, refer to Deb [4].

Among many widely used methods for multi-objective optimization, we introduce the elitist nondominated *sorting genetic algorithm* (NSGA-II) in the rest of this chapter.

14.7 NSGA-II

The elitist *nondominated sorting genetic algorithm*, or NSGA-II for short, was developed by Deb et al. [5] and has become a popular method for solving multi-objective optimization problems by finding multiple Pareto solutions. Its key features include the use of elitist, diversity-preserving mechanisms and emphasis on nondominated solutions.

The main step in NSGA-II is that the n offsprings S_t are created from the n parents P_t using the standard genetic algorithms. The whole population W_t of size $2n$ is formed by joining S_t and P_t, i.e., $W_t = P_t \cup S_t$. Then the nondominated sorting is applied to W_t. Then the new population is filled, one at a time, by different nondominated solutions. Because the population of W_t is $2n$, only half of it will be put into the new population, selecting the nondominated solutions for the Pareto front with the higher diversity while discarding the rest of the solutions.

The crowding distance D_i used in the NSGA-II is essentially based on the cardinality of the solution sets and their distance to the solution boundaries. The crowded tournament selection is based on ranking and distance. In other words, if a solution $\boldsymbol{x}_i$ has a better rank than $\boldsymbol{x}_j$, we select $\boldsymbol{x}_i$. If the ranks are the same but $D_i > D_j$, we select $\boldsymbol{x}_i$ based on its crowding distance. For details, refer to Deb et al. [5]. The main steps of NSGA-II can be summarized as follows:

1. Create a new population $W_t = P_t \cup S_t$ by combining P_t and apply nondominated sorting.
2. Identify different fronts PF_i ($i = 1, 2, \ldots$).
3. Generate P_{t+1} from $P_{t+1} = \emptyset$ with $i = 1$ and fill $P_{t+1} = P_{t+1} \cup PF_i$ until size n.
4. Carry out a crowding sort using the crowd distance to sort some PF_i to P_{t+1}.
5. Generate new offspring S_{t+1} from P_{t+1} via crowded tournament-based genetic operators: crossover, mutation, and selection.

There are many other methods for solving multi-objective optimization problems. Some other evolutionary algorithms or approaches include the vector evaluated genetic algorithm (VEGA) [11], differential evolution for multi-objective optimization (DEMO) [2], and the strength pareto evolutionary algorithm (SPEA) [4].

Recent studies show that other methods such as multi-objective cuckoo search (MOCS), multi-objective firefly algorithm (MOFA), and multi-objective flower pollination algorithms (MOFPA) can be equally or even more effective [15–17].

References

[1] Coello CAC, Pulido GT, Lechuga MS. Handling multiple objectives with particle swarm optimization. IEEE Trans Evol Comput 2004;8(3):256–79.

[2] Chakraborty UK. Advances in differential evolution. Studies in computational intelligence, vol. 143. Heidelberg, Germany: Springer; 2008.

[3] Deb K. Optimization for engineering design: algorithms and examples. New Delhi, India: Prentice-Hall; 1995.
[4] Deb K. Multi-objective optimization using evolutionary algorithms. Chichester, UK: Wiley and Sons; 2001.
[5] Deb K, Agrawal S, Pratap A, Meyarivan T. A fast and elitist multi-objective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 2002;6(2):182–97.
[6] Fonseca CM, Fleming PJ. An overview of evolutionary algorithms in multiobjective optimization. Evol Comput 1995;3(1):1–16.
[7] Haimes YY, Lasdon LS, Wismer DA. On a bicriterion formulation of the problems of integrated system identification and system optimization. IEEE Trans Syst Man Cybern 1971;1(4):296–7.
[8] Knowles JD, Corne DW. Approximating the non-dominated front using the Pareto archived evolution strategy. Evol Comput J 2000;8(2):149–72.
[9] Pareto V. Manuale di economica politica. London, UK: Macmillan; 1972.
[10] Sawaragi Y, Nakayama H, Tanino T. Theory of multiobjective optimization. London, UK: Academic Press; 1985.
[11] Schaffer JD. Multiple objective optimization with vector evaluated genetic algorithms. In: Grefenstette J, editor. Proceedings of the International Conference on Genetic Algorithms and their Applications. Hillsdale, NJ, USA: L. Erlbaum Associates; 1985. p. 93–100.
[12] Srinivas N, Deb K. Multiple objective optimization using nondominated sorting in genetic algorithms. Evol Comput 1994;2(3):221–48.
[13] Suppapitnarm A, Seffen KA, Parks GT, Clarkson PJ. A simulated annealing algorithm for multiobjective optimization. Eng Optim 2000;33(1):59–85.
[14] Talbi E-G. Metaheuristics: from design to implementation. Wiley; 2009.
[15] Yang XS, Deb S. Multiobjective cuckoo search for design optimization. Comput Oper Res 2013;40(6):1616–24.
[16] Yang XS. Multi-objective firefly algorithm for continuous optimization. Eng Comput 2013;29(2):175–84.
[17] Yang XS, Karamanoglu M, He X. Multi-objective flower algorithm for optimization. Procedia Comput Sci 2013;18:861–8.
[18] Zitzler E, Deb K, Thiele L. Comparison of multiobjective evoluationary algorithms: Empirical results. Evol Comput 2000;8(2):173–95.
[19] E Zitzler, M Laumanns, S Bleuler. A tutorial on evolutionary multiobjective optimization. In: Gandibleux X, Sevaux M, Sörensen K, T'kindt V, editors. Metaheuristics for multiobjective optimization. Lecture notes in economics and mathematical systems, vol. 535. Berlin, Germany: Springer; 2004. p. 3–37.